Machine Learning Diploma

DEEP LEARNING 5 :DEEP NEURAL NETWORK

AMIT

Agenda

- Optimization Algorithm
  - Review GD
  - Momentum
  - Adagrad
  - Rmsprop
  - Adam
- Project
- References

Optimization Definition

Optimization

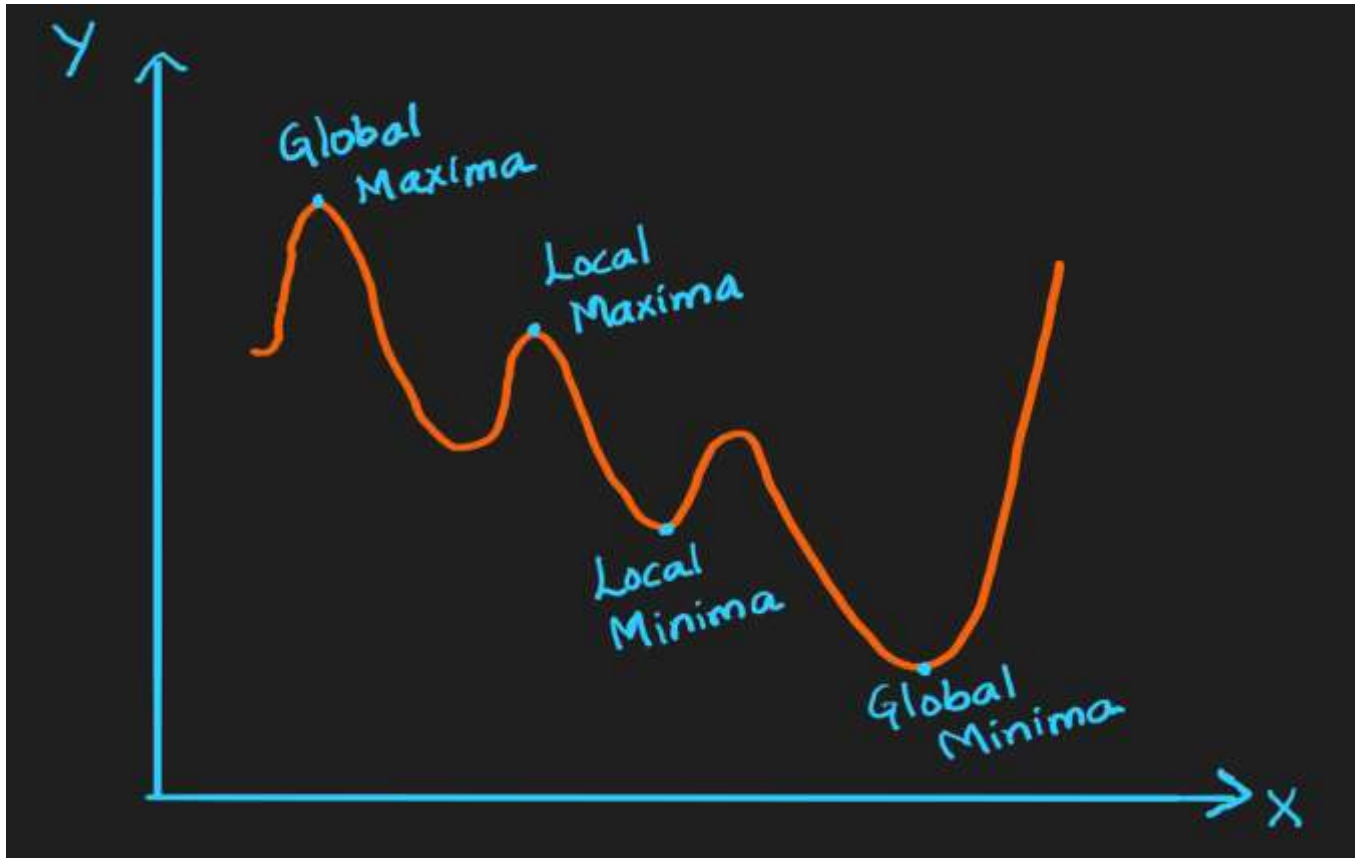
- Definition:
  - The word optimization means to find a set of learning parameters to an objective function that results in either maximizing or minimizing this objective function.
- For any machine learning problem, we usually define a loss function, aka cost function or objective function, that helps in defining what our problem targets. The loss function normally needs to be minimized, however sometimes the loss function can be maximized so we just flip the sign of the function. Here come the question, **how can we minimize the loss?**
  - The answer is simply to apply optimization algorithm iteratively until reaching the best set of learning parameters that minimizes (or maximizes in rare cases) the objective

Optimization

- **MAXIMA AND MINIMA:**
  - Maxima is the largest and Minima is the smallest value of a function within a given .
  - *Global Maxima and Minima*: It is the maximum value and minimum value respectively on the entire domain of the function
  - *Local Maxima and Minima*: It is the maximum value and minimum value respectively of the function within a given range.
  - There can be only one global minima and maxima but there can be more than one local minima and maxima.

Optimization

- **MAXIMA AND MINIMA:**



Optimization

- **How do optimizers work in general? (Analogy)**
  - Think of optimizers as hiker going down a mountain with blindfold on. You can't know which direction to move towards to reach down the mountain. But blindly you can know easily if you are going down or going up the mountain. Hence if you kept moving forward in a way that you are going forward so sooner or later you will reach down the mountain. Moreover if you are blind, you wouldn't be able to know where could be the closest start to reach the base in a fast way but will start from some random point and start going down.
  - Similarly what optimizer does is to start form some random weights and keep using the optimizer (sense of going down) to reduce the loss or in other words going down the convex function until reaching the minima.

Optimization Algorithms

1)Remember : Gradient Descent

- We have taken the most famous optimization algorithm which is Gradient descent. Gradient descent is typically an optimization algorithm where it is based on convex functions that tweaks its parameters iteratively to minimize the objective function using its gradient (differentiable form). Simply it starts with some initialized weights that keeps changing according to the next update equation:

$$\text{Repeat until convergence } \{$$

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

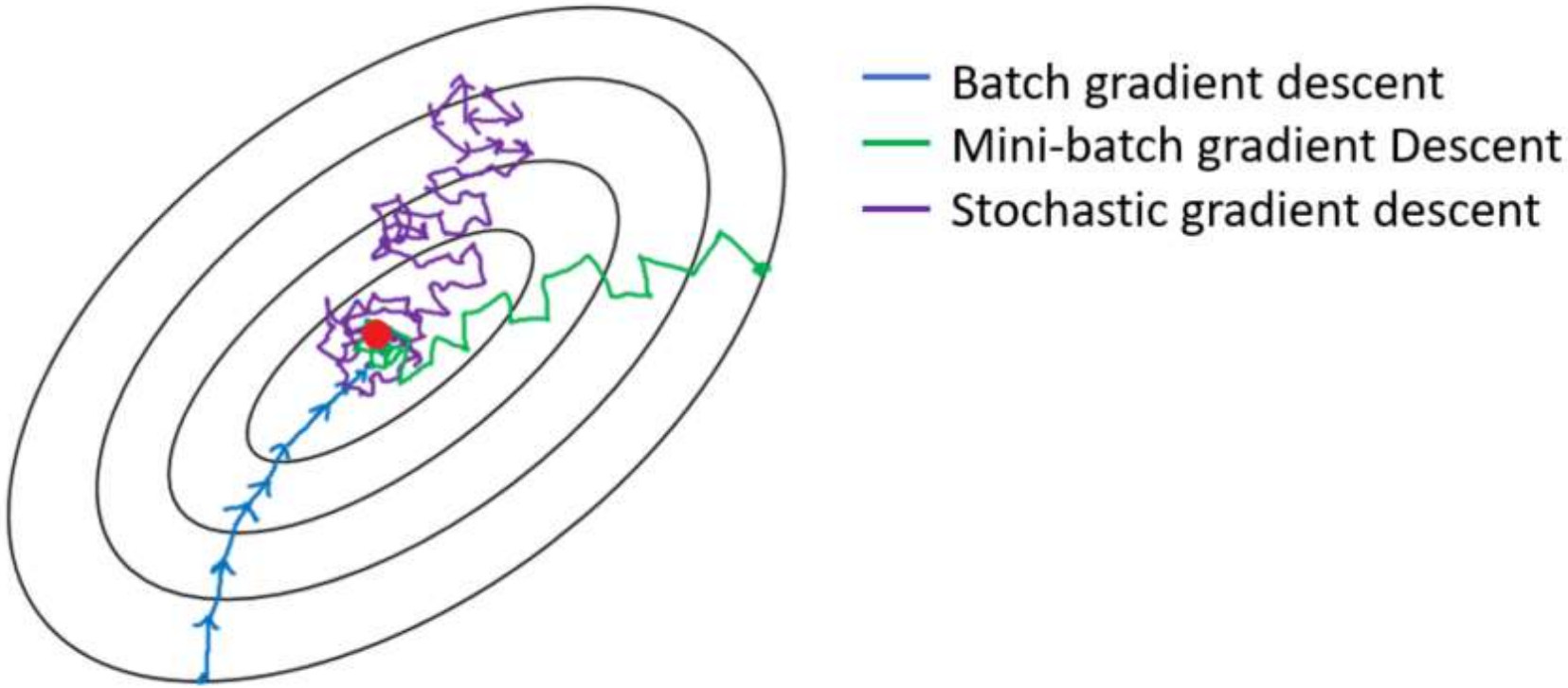
$$\}$$

- As we can notice that there is a learning rate that defines how big our steps will be noting that making it small will lead to very slow learning process while letting it high will lead to oscillations.

1)Gradient Descent

- Batch Gradient Decent
  - BGD is a variation of the gradient descent algorithm that calculates the error for each data point in the training datasets, but only updates the model after all training examples have been evaluated. Takes 1 iteration.
- Stochastic Gradient Descent
  - For an Epoch take a random data point from dataset and calculate its loss then update the weights and repeat over the rest of the data until the epoch is done, it's causes a stochastic conversion but uses less memory. Takes N iterations.
- Mini Batch Gradient descent
  - *MGD is a variation of the gradient descent algorithm that splits the training datasets into small batches that are used to calculate model error and update model coefficients. Takes Number of Batches iterations.*

Gradient Descent



2)Gradient Descent With Momentum

- Let us now think of a ball going down a hill with a gentle slope: it will start at first slowly going down, but it will quickly gets some momentum so its velocity will increase until reaching a final velocity (noting that there is a friction or air resistance). This is called in physics the momentum, so we will try to apply the same idea here in our optimization algorithm.

- SGD was good at being fast optimizers than the batch one and less memory intensive but it injects too much oscillations leading to slower convergence overall. Taking into consideration as well that gradient descent will take regular steps down the slope which will lead to much more time to reach the bottom.

2)Gradient Descent With Momentum

- GD momentum optimization work by taking the previous gradients into our consideration in the update equation which leads to the following equation:

$$w_{t+1} = w_t - \alpha V_t$$

- Instead of using just the gradient of the cost function w.r.t. the learning parameter, we will put the velocity instead. To be exact, this is not the velocity but the exponential weighted average of current and past gradients (up to time t).

$$V_t = \beta V_{t-1} + (1-\beta)\frac{\partial L}{\partial w_t}$$

2)Gradient Descent With Momentum

- The equation is simply taking the larger effect to the most recent updates through multiplying by β compared to the previous ones multiplied by β^n (1-β). If you want to understand how this equation says so, let us do it with numbers.
- Assume β (momentum) is 0.9 and we start from $V_0 = 0$, we can reach the following:

  → $V_1 = 0.9\ V_0 + (1-0.9)*(\text{gradient})$ but V_0 equals 0 so,

  → $V_1 = 0.1$ (gradient at t=1) (1)

- By applying (1) in the update equation, hence $W_1 = W_0 - \alpha\ (0.1 * \text{gradient at } t=1)$ Let us now move to the weight in the next iteration: $V_2 = 0.9\ V_1 + (1-0.9)$ (gradient at t=2) but $V_1 = 0.1$ (gradient at t=1) Then $V_2 = 0.9\ (0.1 * \text{gradient at } t=1) + 0.1$ (gradient at t=2) (2)

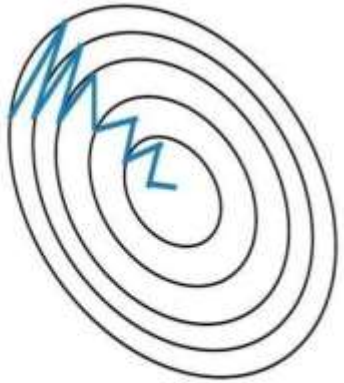
2)Gradient Descent With Momentum

- Momentum was invented for reducing high variance in SGD and softens the convergence (reaching the global minima). It accelerates the convergence towards the relevant direction and reduces the fluctuation to the irrelevant direction by adding the speed parameter, one disadvantage it may overshoot the local minima if Learning rate was too high.

$$v_{dw} = \beta \cdot v_{dw} + (1 - \beta) \cdot dw$$

$$W = W - \alpha \cdot v_{dw}$$

Stochastic Gradient Descent **withhout** Momentum



Stochastic Gradient Descent **with** Momentum

3) Adagrad (Adaptive Gradient)

- The idea behind Adagrad is to use different learning rates for each parameter base on iteration. The reason behind the need for different learning rates is that the learning rate for sparse features parameters needs to be higher compare to the dense features parameter because the frequency of occurrence of sparse features is lower.

  - Equation:

$$\text{SGD} \Rightarrow \; w_t = w_{t-1} - \eta \frac{\partial L}{\partial w_{t-1}}$$

$$\text{Adagrad} \; \Rightarrow w_t = w_{t-1} - \eta_t^{'} \frac{\partial L}{\partial w_{t-1}}$$

$$\text{where } \eta_t^{'} = \frac{\eta}{\sqrt{\alpha_t + \varepsilon}}$$

$$\varepsilon \text{ is a small } +\text{ve number to avoid divisibilty by } 0$$

$$\alpha_t = \sum_{i=1}^{t} \left(\frac{\partial L}{\partial w_{t-1}}\right)^2 \quad \textit{summation of gradient square}$$

3) Adagrad (Adaptive Gradient)

- In the previous Adagrad optimizer equation, the learning rate has been modified in such a way that it will automatically decrease because the summation of the previous gradient square will always keep on increasing after every time step.
- let's take a simple example to check how the learning rate is different for every parameter in a single time step. For this example, we will consider a single neuron with 2 inputs and 1 output. So, the total number of parameters will be 3 including bias.

$$Let \quad w_t^1,\ w_t^2 \text{ be the weight parameter } b_t \text{ be the bias at a given time step}$$

$$\begin{bmatrix} w_t^1 \\ w_t^2 \\ b_t \end{bmatrix} = \begin{bmatrix} w_{t-1}^1 \\ w_{t-1}^2 \\ b_{t-1} \end{bmatrix} - \begin{bmatrix} \frac{\eta}{\sqrt{\alpha_t^1 + \varepsilon}} \\ \frac{\eta}{\sqrt{\alpha_t^2 + \varepsilon}} \\ \frac{\eta}{\sqrt{\alpha_t + \varepsilon}} \end{bmatrix} * \begin{bmatrix} (\frac{\partial L}{\partial w_{t-1}^1}) \\ (\frac{\partial L}{\partial w_{t-1}^2}) \\ (\frac{\partial L}{\partial b_{t-1}}) \end{bmatrix}$$

3) Adagrad (Adaptive Gradient)

- The previous computation is done at a single time step, where all the three parameters learning rate "η" is divided by the square root of "α" which is different for all parameters. So, we can see that the learning rate is different for all three parameters.
- Now, let's see how weights and bias are updated in Stochastic Gradient Descent.

$$Let \quad w_t^1,\ w_t^2 \ be\ the\ weight\ parameter\ b_t\ be\ the\ bias\ at\ a\ given\ time\ step$$

$$\begin{bmatrix} w_t^1 \\ w_t^2 \\ b_t \end{bmatrix} = \begin{bmatrix} w_{t-1}^1 \\ w_{t-1}^2 \\ b_{t-1} \end{bmatrix} - \begin{bmatrix} \eta \\ \eta \\ \eta \end{bmatrix} * \begin{bmatrix} (\frac{\partial L}{\partial w_{t-1}^1}) \\ (\frac{\partial L}{\partial w_{t-1}^2}) \\ (\frac{\partial L}{\partial b_{t-1}}) \end{bmatrix}$$

3) Adagrad (Adaptive Gradient)

- Similarly, the above computation is done at a single time step, and here the learning rate “η” remains the same for all parameters.
- Lastly, despite not having to manually tune the learning rate **there is one huge disadvantage** ; due to monotonically decreasing learning rates, at some point in time step, the model will stop learning as the learning rate is almost close to 0.

4) Rmsprop(Root Mean Squared Propagation

- As we've seen, AdaGrad runs the risk of slowing down a bit too fast and never converging to the global optimum. The Rmsprop algorithm fixes this by accumulating only the gradients from the most recent iterations (as opposed to all the gradients since the beginning of training). It does so by using exponential decay (exponentially weighted moving average).

- Update equation will be the same:

$$w_{t+1} = w_t - \frac{\alpha}{\sqrt{S_t + \epsilon}} \cdot \frac{\partial L}{\partial w_t}$$

- However S_t now is defined as exponential moving average of squared gradients:

$$S_t = \beta S_{t-1} + (1 - \beta) \left[\frac{\partial L}{\partial w_t} \right]^2$$

4) Rmsprop(Root Mean Squared Propagation

- Rmsprop solves the overshooting problem and modify its path to the global minima.
- Ex initialization, Learning rate =0.01, beta =0.9, v starts with0 and epsilon =0.000001.

$$v_{dw} = \beta \cdot v_{dw} + (1 - \beta) \cdot dw^2$$

$$W = W - \alpha \cdot \frac{dw}{\sqrt{v_{dw}} + \epsilon}$$

5) Adam(Adaptive Momentum Estimation)

- Adam as combining the advantages of two other extensions of stochastic gradient descent. Specifically:
  - **Adaptive Gradient Algorithm** (AdaGrad) that maintains a per-parameter learning rate that improves performance on problems with sparse gradients (e.g. natural language and computer vision problems).
  - **Root Mean Square Propagation** (RMSProp) that also maintains per-parameter learning rates that are adapted based on the average of recent magnitudes of the gradients for the weight (e.g. how quickly it is changing). This means the algorithm does well on online and non-stationary problems (e.g. noisy).

- $m_t = \beta_1 m_t + (1 - \beta_1) g_t$
- $v_t = \beta_1 m_t + (1 - \beta_1) g_t^2$
- $\widehat{m_t} = \frac{m_t}{1-\beta_1^t}$

To correct for bias of averaging in a 0 at the beginning.

- $\widehat{v_t} = \frac{v_t}{1-\beta_1^t}$

This is like a combination of RMSprop and momentum.

- $w_{t+1} = w_t - \frac{\eta}{\sqrt{\widehat{v_t}}} \widehat{m_t}$

5) Adam(Adaptive Momentum Estimation)

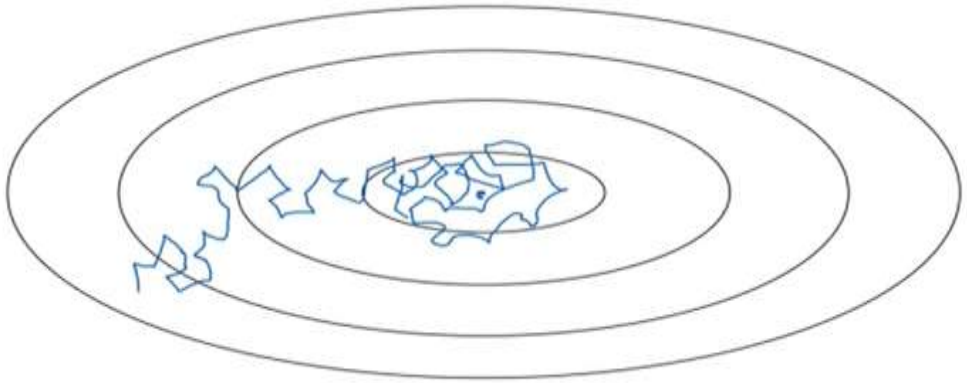
- The momentum decay hyperparameter β_1 is typically initialized to 0.9, while the scaling decay hyperparameter β_2 is often initialized to 0.999. As earlier, the smoothing term ε is usually initialized to a tiny number such as 10^{-7} .

- Since Adam is an adaptive learning rate algorithm (like AdaGrad and RMSProp), it requires less tuning of the learning rate hyperparameter α. You can often use the default value $\alpha = 0.001$, making Adam even easier to use than Gradient Descent.

- Adam is believed to be the fastest algorithm to converge to minima.
- Adam is considered the best algorithm amongst all the algorithms discussed so far.

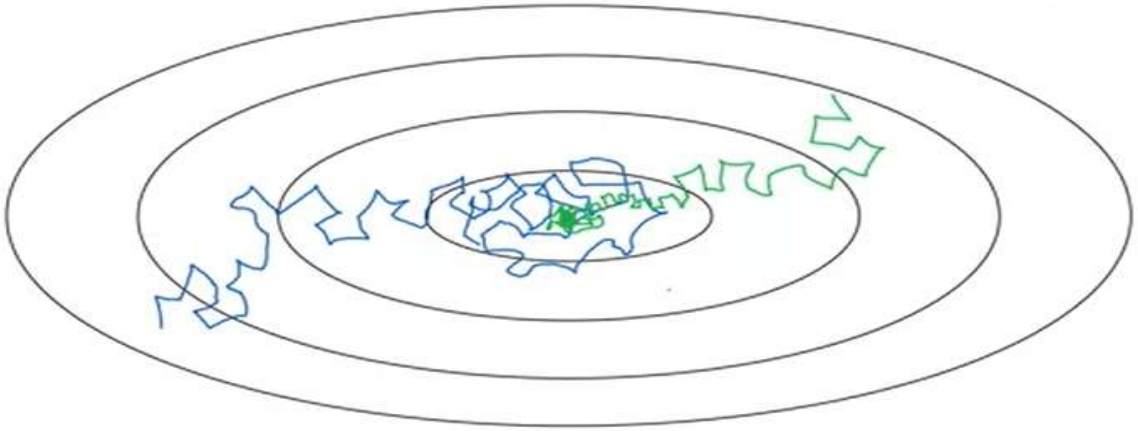
6)Learning Rate Decay

- Learning rate decay is a technique for training modern neural networks. It starts training the network with a large learning rate and then slowly reducing/decaying it until local minima is obtained. It is empirically observed to help both optimization and generalization.



- Algorithm converging with a constant learning rate (Noisy and represented with Blue)

6)Learning Rate Decay



- Algorithm converging while decaying Learning Rate over time (less noisy and represented with green)

6)Learning Rate Decay (Common Method)

“ $\alpha=(1/(1+decayRate\times epochNumber))*\alpha 0$ ”

- *1 epoch : 1 pass through data*
- *α : learning rate (current iteration)*
- *α0 : Initial learning rate*
- *decayRate : hyper-parameter for the method*

6)Learning Rate Decay (Common Method)

- **Exponential Decay**

“ α = (decayRate^(epochNumber))*α0 “

- This function applies an exponential decay function to a provided initial learning rate so that learning rate decay over time , exponentially.
- The decayRate of this method is always less then 1 , 0.95 is most commonly used among practitioners.
- **Epoch Number Based**

“ α = (k/ √epochNumber)* α0 ”

- In this method we take some constant ‘k’ and divide it with square root of epoch number.

End-to-End Project

Any Question?

THANK YOU!

AMIT